SPIT Cache Module — 100 Technical Viva Questions & Answers

Master Viva Defense & Examination Guide — Covering Phase 2 (Frontend) & Phase 3 (Backend Persistent Core)

Institution: Sardar Patel Institute of Technology (SPIT), Andheri (W), Mumbai
Department: Computer Science & Engineering | **Academic Year:** 2026–27 | **Subject:** Mini Project (PR1)
Student Authors: Jay Shinde (2024800115), Nikhil Shelar (2024800113), Sanchit Gaikwad (2024800029)
Faculty Guide: Prof. Suhas Kakade

Examination Overview: This evaluation handbook comprises exactly 100 targeted technical questions and verified model answers spanning 17 core Computer Science and Software Engineering competencies assessed during the SPIT Mini Project examination. Questions address Phase 2 (Interactive React Telemetry Console, SVG charting, Dark/Light Themes, Command Palette) and Phase 3 (Modular Node.js Backend, O(1) Hash Map & Doubly-Linked List, Write-Ahead Logging with IEEE 802.3 CRC32, Point-in-Time Snapshot Checkpoints, Cold Recovery Pipeline, Server-Sent Events, Cryptographic SHA-256 Audit Chaining, and Sliding-Window Rate Limiting).

1. Architecture & System Design

Q1: How does the SPIT Cache Module achieve loose coupling between the caching core and storage engine?

Answer: The architecture implements the Facade and Dependency Injection patterns. `CacheEngine` coordinates memory lookups without coupling itself directly to low-level POSIX file handles. Durability logic is encapsulated inside `WalManager` and `SnapshotStore`. If disk logging is swapped or disabled, the core in-memory engine executes without code modification.

Q2: What is the architectural role of the Memento pattern in the snapshot subsystem?

Answer: The Memento pattern externalizes internal state without violating encapsulation. `SnapshotStore` captures an immutable, point-in-time state representation of non-expired entries and their temporal metadata, allowing the `RecoveryManager` to restore the engine to that exact checkpoint state later.

Q3: Explain how the single-threaded event loop in Node.js prevents race conditions in cache pointer manipulation.

Answer: Because JavaScript execution in Node.js runs on a single-threaded event loop, synchronous operations like hash map lookup and doubly-linked list node splicing execute non-preemptively. No two operations can manipulate `node.prev` and `node.next` concurrently, eliminating the need for mutexes in memory.

Q4: What is the CAP theorem classification of this single-node persistent cache?

Answer: As a standalone persistent single-node cache, it provides **CP (Consistency and Partition-Tolerance)** characteristics locally. Every write is synchronously committed to disk before returning success, guaranteeing strong read-your-writes consistency across crashes.

Q5: How does the architecture avoid the "Cache Stampede" (Thundering Herd) problem upon restart?

Answer: By executing the two-phase cold recovery pipeline on boot before opening the HTTP port to incoming traffic, the cache is fully pre-warmed with all active keys from the snapshot and WAL. The underlying database is never subjected to a sudden flood of cache misses.

Q6: Why is memory-mapped file I/O (mmap) considered for future architectural revisions?

Answer: Memory-mapped I/O maps the WAL file directly into the process's virtual address space. This avoids copying data between user space and kernel space buffers (`read()` / `write()` syscall overhead), reducing latency for write-intensive operations.

Q7: Describe the component lifecycle during an ungraceful process termination (SIGKILL).

Answer: The volatile in-memory heap is immediately wiped by the operating system kernel. However, all mutations acknowledged prior to the crash have already been written and flushed via `fsync` into `wal.log`. On the next startup, the recovery coordinator detects the log and reconstitutes state.

2. Frontend Architecture & UI/UX

Q8: Why was Vanilla CSS with CSS custom properties chosen over TailwindCSS or UI component libraries?

Answer: Vanilla CSS provides zero-dependency build efficiency, eliminates bundle bloat, and provides granular control over design tokens. CSS custom properties allow instant, flicker-free switching between Obsidian Dark and Clean Slate Light themes without re-rendering the React component tree.

Q9: How is the Global Command Palette (Cmd+K) implemented to ensure keyboard accessibility?

Answer: A root `useEffect` attaches a `keydown` listener to `window`, checking for `(e.metaKey || e.ctrlKey) && e.key === 'k'`. It prevents the browser default search shortcut, traps focus inside the modal, allows arrow-key navigation through options, and closes on `Escape`.

Q10: How does the CacheTable implement client-side multi-field filtering without performance lag?

Answer: Filtering and sorting are wrapped in React `useMemo` hooks. Search queries are checked against lowercase key and value strings, followed by status checking (`ACTIVE` vs `EXPIRED`), column sorting, and array slicing (`slice(startIndex, endIndex)`) to render only the active 10 items.

Q11: How do the SVG charts dynamically draw line curves without external libraries like Chart.js?

Answer: Custom SVG `<path>` elements compute cubic Bezier coordinates from normalized data arrays: x = \frac{i}{N-1} \times \text{width}, y = \text{height} - (\frac{\text{val} - \text{min}}{\text{max} - \text{min}} \times \text{height}). SVG linear gradients provide the glow fill underneath the line.

Q12: How is state persistence achieved for theme preferences across browser reloads?

Answer: When the theme toggle is clicked, the new value (`dark` or `light`) is saved to `localStorage.setItem('spit_cache_theme', theme)` and applied to `document.documentElement.setAttribute('data-theme', theme)`. An inline script in `index.html` reads this before React mounts to avoid theme flickering.

Q13: What prevents memory leaks when subscribing to real-time events in React?

Answer: Event listeners and intervals return cleanup functions in their respective `useEffect` hooks: `return () => { clearInterval(timer); sse.close(); }`. This ensures timers and socket connections are torn down when components unmount.

3. Backend Engineering & Modular Design

Q14: Explain the internal folder structure of Phase 3 and why separation of concerns was emphasized.

Answer: The backend is structured into domain-specific modules: `engine/` (memory structures), `wal/` (disk logging), `snapshot/` (checkpoints), `recovery/` (reconstitution), `events/` (pub-sub), `audit/` (compliance), and `utils/` (CRC32). This modularity allows isolated unit testing and independent evolution of subsystems.

Q15: How does the backend prevent unhandled promise rejections from crashing the process?

Answer: All route handlers in `server.js` are wrapped in `try...catch` blocks. Uncaught exceptions return a standardized JSON error response (`{ error: "Internal Server Error", message: err.message }`) with an HTTP 500 status code.

Q16: How is configuration loaded in the Phase 3 backend?

Answer: The engine reads `Phase_3/config/default.json` at boot, merging it with sensible hardcoded defaults. This allows operators to tune capacity, TTL sweeper intervals, and file paths without changing code.

Q17: Why are ES Modules (`import/export`) used instead of CommonJS (`require`) in the backend?

Answer: ES Modules represent the official ECMAScript standard. They enable static module analysis, tree-shaking, better compatibility with modern frontend frameworks, and clean asynchronous module loading.

Q18: What is the role of `setImmediate()` in the EventBus implementation?

Answer: `setImmediate()` queues event callback execution into the "check" phase of the Node.js event loop, immediately after I/O callbacks. This ensures event processing is asynchronous and does not delay the current HTTP request-response cycle.

Q19: How does `AuditLogger` prevent disk write deadlocks?

Answer: It uses synchronous append operations (`fs.appendFileSync`) wrapped in localized exception handlers. Because operations append small single-line strings, OS page caching handles the write rapidly without blocking.

Q20: How is the Node.js process gracefully shut down on SIGINT or SIGTERM?

Answer: Signal handlers can trigger a clean snapshot checkpoint, flush the active WAL file descriptor (`walManager.close()`), stop the TTL interval sweeper, and exit with code 0.

4. REST APIs & Telemetry Streaming

Q21: What is the difference between GET /api/cache and GET /api/cache/:key?

Answer: `GET /api/cache` returns a paginated metadata list of all keys, values, and expiration states without altering access ordering. `GET /api/cache/:key` simulates a cache read: if found and valid, it updates the LRU position to `HEAD`, increments `accessCount`, and emits `CACHE_HIT`.

Q22: Why is Server-Sent Events (SSE) better suited than WebSockets for our telemetry pipeline?

Answer: Telemetry monitoring is primarily unidirectional (server pushing events and metrics to the client). SSE operates natively over standard HTTP/1.1 or HTTP/2, requires no special connection handshakes, supports automatic browser reconnection, and works cleanly through proxies and firewalls.

Q23: How does the server handle Cross-Origin Resource Sharing (CORS)?

Answer: The server handles `OPTIONS` pre-flight requests by returning HTTP 204 with headers: `Access-Control-Allow-Origin: *`, `Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`, and `Access-Control-Allow-Headers: Content-Type, Authorization`.

Q24: What payload validation occurs when calling POST /api/cache?

Answer: The server verifies that `body.key` exists and is a non-empty string, parses `ttlSeconds` as a non-negative number, and ensures the payload does not exceed the 10 MB threshold.

Q25: What does the POST /api/wal/snapshot endpoint return?

Answer: It returns `{ checkpointLsn, entryCount, timestamp, checksum, fileSizeBytes }`, confirming that all active keys were written to `snapshot.bin` and the WAL was compacted up to that LSN.

Q26: What is the purpose of the POST /api/demo/run-scenario endpoint?

Answer: It executes pre-packaged demonstration workflows (`burst`, `lru`, `ttl`) on the backend, enabling seamless, reproducible demonstrations during academic evaluations.

5. Database & File Storage Schematics

Q27: What is the on-disk format of `wal.log`?

Answer: It is an append-only JSON Lines (`.jsonl`) file where each line represents an atomic transaction containing `magic`, `lsn`, `timestamp`, `opcode`, `key`, `value`, `ttlSeconds`, and `crc32`.

Q28: Why is JSON Lines chosen over a raw binary byte stream for the Phase 3 WAL?

Answer: JSON Lines allows easy human inspection and debugging via standard terminal utilities (`cat`, `grep`, `tail -f`) while remaining simple to parse sequentially. The architecture is modular, allowing conversion to a packed binary format via `Buffer.alloc()` in Phase 4.

Q29: How does the snapshot file differ in structure from the WAL file?

Answer: The WAL is an unbounded append-only log of delta operations (`SET`, `DEL`). The snapshot file is a bounded, point-in-time state checkpoint containing only the active, unexpired keys and their values at a specific LSN.

Q30: Why is `audit.log` stored separately from `wal.log`?

Answer: `wal.log` is a storage-engine log subject to truncation and compaction during snapshotting. `audit.log` is an immutable, permanent compliance log tracking which actor triggered operations, and must never be truncated.

Q31: What permissions should be applied to the `Phase_3/database/` directory in production?

Answer: Strict POSIX permissions `chmod 700` for the directory and `chmod 600` for the log files, ensuring that only the cache engine process user can read or modify persistent state.

Q32: What occurs if the persistent disk partition runs out of space?

Answer: Synchronous disk appends throw `ENOSPC` errors. The engine halts state mutations to prevent inconsistency, emits an emergency telemetry event, and serves reads from in-memory structures until disk space is reclaimed.

6. Algorithms & Complexity Analysis

Q33: Prove why the DoublyLinkedList `addToHead` operation is strictly O(1).

Answer: `addToHead(node)` modifies four pointers: `node.prev = head`, `node.next = head.next`, `head.next.prev = node`, `head.next = node`. None of these operations depend on the number of elements N in the list, satisfying the definition of O(1) constant time.

Q34: What is the worst-case time complexity of cache lookup in standard hash indexing?

Answer: While average time complexity is O(1), hash collisions can theoretically cause worst-case degradation to O(N) if all keys hash to the same bucket. Modern hash map implementations mitigate this by using balanced red-black trees for collided buckets, maintaining O(\log N) worst-case bounds.

Q35: Explain the time and space complexity of the active TTL sweeper.

Answer: Given a sample size K (e.g., K = 10), the time complexity is O(K), which is O(1) constant time relative to total cache size N. Space complexity is O(1) since only K keys are sampled into memory.

Q36: Why is LRU eviction preferable over FIFO eviction for caching workloads?

Answer: FIFO evicts keys based strictly on creation time, frequently discarding hot keys that are queried continuously. LRU tracks access frequency recency, retaining frequently accessed hot keys at the head of the list and maximizing the cache hit ratio.

Q37: What is the time complexity of scanning the WAL file during cold restart?

Answer: O(W), where W is the number of un-compacted WAL records written since the last snapshot checkpoint. Because records are scanned sequentially without random seeks, disk throughput reaches sequential read speeds (500+\text{MB/s} on SSDs).

Q38: What is Belady's Anomaly and does the LRU algorithm suffer from it?

Answer: Belady's Anomaly is a phenomenon where increasing cache capacity results in an increased number of cache misses (common in FIFO). LRU belongs to the class of **stack algorithms**, mathematically proving that it is immune to Belady's Anomaly.

Q39: How does LFU (Least Frequently Used) compare algorithmically with LRU?

Answer: LFU evicts keys with the lowest access count. While optimal for long-term stable popularity distributions, it suffers from "cache pollution" where historically popular keys remain cached long after they become obsolete. LRU adapts faster to shifting access patterns.

Q40: What is the space overhead of maintaining Doubly-Linked List nodes?

Answer: Each node requires approximately 48–64 bytes for object headers and pointers (`prev`, `next`, `key`, `value`, `createdAt`, `expiresAt`, `accessCount`). For 10,000 keys, total node pointer overhead is under 1\text{MB}, making it negligible compared to stored data payloads.

7. Cache Management & Eviction Policies

Q41: What happens when an existing key is inserted again with a different value and TTL?

Answer: The engine locates the node via the hash map in O(1), updates `node.value`, recalculates `node.expiresAt`, increments `accessCount`, splices the node to `HEAD`, and logs an `OP_UPDATE` record to the WAL.

Q42: Can an entry be configured with an infinite lifespan?

Answer: Yes. Setting `ttlSeconds = 0` sets `expiresAt = 0`. The expiration checks treat zero as infinite duration, ensuring the key is only removed via explicit deletion or capacity-triggered LRU eviction.

Q43: What is the difference between cache capacity and memory capacity?

Answer: Cache capacity limits the maximum number of distinct keys stored (e.g., 50 entries). Memory capacity limits total heap bytes occupied. SPIT Cache enforces entry capacity directly and monitors heap byte utilization via metrics.

Q44: How does the cache engine handle deletion of non-existent keys?

Answer: It queries `index.has(key)`. If false, it returns `false` without writing an unnecessary tombstone record to the WAL, conserving disk I/O.

Q45: What is a tombstone record in write-ahead logging?

Answer: A tombstone is an append-only WAL record with opcode `DEL` indicating that a key has been deleted. During recovery replay, encountering this tombstone instructs the recovery engine to remove that key from memory.

Q46: How does the system ensure expired keys do not return stale data?

Answer: The passive lazy check inside `get()` compares `Date.now() >= node.expiresAt` before returning any value. If true, the key is purged on the spot and a cache miss is returned.

Q47: Why is random sampling used in the active TTL sweeper instead of a sorted priority queue?

Answer: A min-heap priority queue ordered by expiration time incurs O(\log N) insertion and deletion overhead on every cache write. Random sampling executes in O(1) time while statistically cleaning expired entries before memory exhaustion occurs (the same strategy employed by Redis).

8. Durability & Persistence Mechanics

Q48: What does the ACID durability property guarantee in this caching module?

Answer: It guarantees that once a `PUT` or `DELETE` operation returns a success response to the client, the transaction is committed to non-volatile disk storage and will survive any subsequent process crash or power loss.

Q49: What is the difference between asynchronous buffer flushing and synchronous fsync?

Answer: Standard file writes place data into the OS kernel page cache. If the operating system halts or loses power before flushing to disk, unwritten data is lost. Synchronous `fdatasyncSync` forces the kernel to flush dirty blocks to disk media before returning control.

Q50: What is a Log Sequence Number (LSN) and how does it advance?

Answer: The LSN is a 64-bit integer initialized at startup from the existing log. Every mutation atomically increments `currentLsn += 1`. Each WAL record and snapshot checkpoint is tagged with its corresponding LSN.

Q51: What is the Recovery Point Objective (RPO) when `syncToDisk` is enabled?

Answer: RPO = 0. Zero acknowledged transactions are lost.

Q52: What is the difference between Write-Through and Write-Ahead caching?

Answer: In Write-Through, writes update the cache and the primary persistent database simultaneously. In Write-Ahead caching, mutations are appended to an append-only transaction log first, and cache state is updated immediately in memory.

Q53: How does log compaction prevent the WAL file from consuming all disk space?

Answer: When a snapshot is created, it writes the complete active dataset at LSN K. The WAL manager then safely removes all records with LSN \le K, because their net effect is already captured within the snapshot.

9. Write-Ahead Logging (WAL) & Integrity

Q54: How is the CRC32 checksum computed on each WAL record?

Answer: A canonical payload string is assembled: `magic|lsn|timestamp|opcode|key|JSON(value)|ttlSeconds`. This string is converted to a UTF-8 buffer and passed to `computeCRC32Hex()`, producing an 8-character hexadecimal string stored in the `crc32` field.

Q55: What specific failure mode does CRC32 detect during log replay?

Answer: Torn writes (partial writes) caused by sudden power loss mid-flush, storage bit-rot, and unauthorized manual tampering with the log file.

Q56: What happens if an operator manually edits a line in `wal.log` using a text editor?

Answer: Modifying characters alters the payload bytes without updating the CRC32 trailer. During cold recovery, `record.verifyChecksum()` evaluates to `false`, isolating the corrupted record and preventing corrupted state ingestion.

Q57: Why is CRC32 used instead of cryptographic hashes like SHA-256 for WAL integrity?

Answer: CRC32 is designed specifically for hardware error and data corruption detection. It computes substantially faster than cryptographic hashes, achieving throughputs exceeding gigabytes per second while providing sufficient error detection for disk blocks.

Q58: What are the magic bytes at the beginning of each WAL record?

Answer: The two-character string `'SP'` (identifying Sardar Patel Institute of Technology). Magic bytes verify that the file being parsed is a genuine SPIT Cache log before parsing payloads.

Q59: Can WAL records be read concurrently while the engine is writing new entries?

Answer: Yes. Because file appending only writes new bytes at the end of the file descriptor, readers scanning existing bytes up to the verified length do not interfere with append operations.

Q60: How does the system determine the last valid LSN on boot if the server crashed mid-write?

Answer: `WalManager.scanCurrentLsn()` reads the file backwards from the end, locating the last line that deserializes into a valid record with a matching CRC32 checksum.

10. Crash Recovery Subsystems

Q61: What are the two distinct phases of the cold-start crash recovery algorithm?

Answer: Phase 1 restores point-in-time state from `snapshot.bin` up to `checkpointLsn`. Phase 2 sequentially replays mutations from `wal.log` forward where LSN > checkpointLsn, applying them to the in-memory cache.

Q62: What happens during recovery if the snapshot file is missing?

Answer: The recovery manager sets `checkpointLsn = 0` and proceeds directly to Phase 2, replaying all valid records from the beginning of `wal.log`.

Q63: What happens during recovery if the snapshot file exists but its checksum fails?

Answer: The recovery manager logs a corruption alert, rejects the corrupted snapshot, resets `checkpointLsn = 0`, and falls back to complete WAL replay.

Q64: How does the RecoveryManager handle bad blocks or partial writes at the end of the WAL?

Answer: It treats the first checksum mismatch as the crash boundary. It logs the corrupted LSN, isolates the corrupted record, stops further replay, and reports `corruptedRecordsIsolated: 1` in the recovery summary.

Q65: If a key was inserted in the snapshot and later deleted in the WAL, what is its state after recovery?

Answer: The key is initially loaded during Phase 1. In Phase 2, the replay engine processes the `DEL` record from the WAL and unlinks the key from memory. The final recovered state correctly reflects that the key does not exist.

Q66: How does the RecoveryManager update the Log Sequence Number pointer?

Answer: After processing valid records, it sets `walManager.currentLsn = summary.lastRecoveredLsn`, ensuring subsequent writes continue sequentially without colliding with historical LSNs.

Q67: How can evaluators test crash recovery interactively?

Answer: Via the web console Recovery page or `POST /api/recovery/crash-simulate` (wiping memory), followed by clicking `Execute Cold Recovery` to observe all keys reconstituted with identical TTLs and values.

11. Event-Driven Pipeline & Decoupling

Q68: What is the Observer pattern and how does `EventBus` implement it?

Answer: The Observer pattern defines a one-to-many dependency where a subject notifies observers of state changes. `EventBus` maintains an event emitter where internal modules (audit logger, SSE streamer, metrics counters) register callback listeners without the cache engine needing direct knowledge of them.

Q69: What parameters are included in a standard `CacheEvent` payload?

Answer: Unique event ID (`EVT-<timestamp>-<seq>`), event type (`CACHE_PUT`, `CACHE_HIT`, etc.), epoch timestamp, and operational metadata (`key`, `value`, `lsn`, `accessCount`).

Q70: How does the server push real-time events to the frontend without polling?

Answer: Clients establish a persistent connection to `/api/events/stream`. The server keeps the HTTP response socket open and writes `data: JSON(event)\n\n` whenever the `EventBus` emits a new event.

Q71: What is the maximum history buffer size of the EventBus in Phase 3?

Answer: Configured to 500 events by default. New events push older entries out of the array, preventing unbounded memory growth.

Q72: Why are event IDs structured with a timestamp and sequence counter?

Answer: `EVT-{Date.now()}-{sequence}` guarantees chronological sortability and uniqueness even when multiple events occur within the same millisecond.

Q73: How does the event pipeline support offline auditing?

Answer: The `AuditLogger` subscribes to administrative actions, serializing records synchronously to `Phase_3/database/audit.log` for permanent compliance storage.

12. Testing & Quality Assurance

Q74: What test runner is used for Phase 3 and why?

Answer: A custom native Node.js test runner using `node:assert`. It runs in sub-second time without external test framework overhead (Jest/Mocha), making it universally executable on any environment with Node.js installed.

Q75: What 4 test suites are executed in `Phase_3/tests/run_tests.js`?

Answer: 1. Doubly-Linked List & Sentinel Node Splicing 2. In-Memory Cache Engine & LRU Eviction Under Capacity 3. Write-Ahead Logging & CRC32 Data Integrity 4. Point-in-Time Snapshotting & Two-Phase Crash Recovery

Q76: How does Suite 2 verify LRU eviction correctness?

Answer: It initializes capacity to 3, inserts keys 1, 2, 3, accesses key 1 (making 1 MRU and 2 LRU), and inserts key 4. It asserts that key 2 is evicted (`get('user:2') === null`) while keys 1, 3, and 4 remain present.

Q77: How does Suite 3 verify CRC32 tampering detection?

Answer: It creates a `WalRecord`, verifies `verifyChecksum() === true`, mutates `record.value = 'tampered'`, and asserts that `verifyChecksum() === false`.

Q78: How does Suite 4 verify cold-start crash reconstitution?

Answer: It saves a snapshot, adds an extra key, calls `engine.clearDirectly()` to simulate a crash, runs `recoveryManager.performRecovery()`, and asserts that both snapshot and post-snapshot keys are restored.

Q79: What was the pass percentage of the automated Phase 3 test suite?

Answer: 100\% pass rate (9 out of 9 unit and integration tests passed).

13. Security & Access Controls

Q80: How does the server prevent Denial-of-Service attacks through huge JSON payloads?

Answer: `parseJsonBody()` tracks the cumulative length of data chunks. If `body.length > 10 *1024* 1024` (10 MB), it destroys the stream and rejects the request with an error.

Q81: How are injection attacks prevented in key and value handling?

Answer: Keys are treated strictly as string literals and stored in a hash map without SQL or shell command execution. Values are sanitized via standard JSON serialization.

Q82: What security risks arise from leaving the telemetry dashboard unauthenticated?

Answer: In production, open management consoles allow unauthorized users to wipe cache entries or trigger recovery simulations. Production deployments require reverse proxy authentication (e.g., NGINX with HTTP Basic Auth, OAuth2, or JWT middleware).

Q83: How does the application prevent Path Traversal attacks when writing database files?

Answer: File paths are resolved via `path.resolve()` and hardcoded configuration paths relative to the application directory, preventing user input from traversing directories via `../`.

Q84: Why are sensitive values like passwords unsuitable for unencrypted cache storage?

Answer: Unencrypted cache payloads stored in memory or in plaintext `wal.log` files could be read via memory dumps or unauthorized file access. Sensitive credentials must be hashed (bcrypt/Argon2) or encrypted before caching.

14. Performance Profiling & Scalability

Q85: What is the typical in-memory lookup latency of this cache engine?

Answer: Under 5 microseconds (< 0.005\text{ms}) on standard desktop hardware, as it requires only a hash table index lookup and updating pointers in the doubly-linked list.

Q86: How does the hit rate affect overall application response times?

Answer: If the database query latency is 20\text{ms} and cache latency is 0.05\text{ms}, a 90\% hit rate reduces average latency from 20\text{ms} to 0.9(0.05) + 0.1(20) = 2.045\text{ms} (a \approx 10\times speedup).

Q87: What is the memory footprint of storing 1,000 key-value pairs?

Answer: Approximately 200–500 KB depending on payload size, including 128 bytes of structural node overhead per entry.

Q88: What is the primary bottleneck when `syncToDisk` is enabled?

Answer: The IOPS (Input/Output Operations Per Second) and write latency of the underlying physical storage drive, as each write calls `fdatasyncSync()`.

Q89: How can write throughput be optimized while maintaining near-zero data loss?

Answer: Group commit (micro-batching). Instead of calling `fsync` on every individual write, collect writes over a 1–5 millisecond window and flush them in a single batched disk sync.

15. Git Workflows & Version Control

Q90: What branching strategy was adopted for Phase 3 and why?

Answer: A Git Flow variant with `main` (stable production releases), `develop` (integration), and dedicated `feature/*` and `bugfix/*` branches. This prevents unstable code from contaminating the main evaluation branch.

Q91: Give examples of Conventional Commits used during development.

Answer: - `feat(engine): implement doubly-linked list LRU eviction` - `feat(wal): add CRC32 checksum serialization` - `fix(recovery): resolve bad block isolation boundary` - `docs(phase3): write 100 viva questions and answers`

Q92: What branch protection rules are recommended for `main`?

Answer: Require pull request reviews before merging, require automated status checks (passing test suite) to pass, and disallow force pushes (`git push --force`).

Q93: How does Git track changes in persistent database files like `wal.log`?

Answer: Real data directories containing `wal.log` and `snapshot.bin` should be added to `.gitignore` to prevent tracking volatile binary test files in version control, while checking in documentation and source code.

Q94: What is the difference between `git merge` and `git rebase` in feature branch workflows?

Answer: `git merge` preserves full branching history by creating a two-parent merge commit. `git rebase` rewrites commit history onto the tip of the target branch, producing a linear, clean commit sequence.

16. Phase 3 Module Implementation

Q95: How do Phase 2 and Phase 3 differ in architectural maturity?

Answer: Phase 2 focused on frontend telemetry visual design, mock orchestration, and architectural modeling. Phase 3 implements the complete, independently runnable backend with real POSIX disk I/O, synchronous WAL appends, CRC32 calculations, snapshot storage, and automated two-phase crash recovery.

Q96: How can the Phase 3 backend and frontend be verified independently?

Answer: The backend can be started with `node Phase_3/backend/src/server.js` and queried directly using `curl` or automated tests (`node Phase_3/tests/run_tests.js`). The frontend can be started with `npm run dev` in `Phase_3/frontend`, communicating over HTTP REST with the backend.

Q97: What happens if the Phase 3 frontend runs while the backend is offline?

Answer: The frontend includes built-in fallback resilience. If `http://localhost:8080` is unreachable, it logs a warning and serves demo data locally so that user interface demonstrations never crash during evaluations.

17. Phase 4 Clustering & Future Scope

Q98: What is the Raft Consensus Algorithm and why is it planned for Phase 4?

Answer: Raft is an understandable consensus algorithm for managing a replicated log across multiple servers. In Phase 4, Raft will replicate the SPIT Cache WAL across 3 nodes, ensuring automatic leader election and zero data loss even if an entire physical server node fails.

Q99: How will consistent hashing be implemented for horizontal sharding in Phase 4?

Answer: Keys will be hashed onto a 32-bit circular ring (0 \text{ to } 2^{32}-1). Each physical node is assigned multiple virtual positions on the ring. Lookups route to the first node encountered moving clockwise, ensuring that adding or removing a node requires migrating only \frac{K}{N} keys.

Q100: How will the Redis RESP3 protocol adapter work in Phase 4?

Answer: A raw TCP socket listener on port 6379 will decode Redis serialization frames (simple strings `+`, errors `-`, integers `:`, bulk strings `$`, arrays `*`). Standard Redis client libraries (Python `redis`, Java Jedis, Node `ioredis`) will be able to query the SPIT Cache directly without needing custom HTTP client libraries.